

DARUKAA.EARTH

15-Hour Production-Ready Hackathon Roadmap

Objective: Build a polished full-stack geospatial carbon & biodiversity project dashboard that satisfies the supplied challenge while maximizing what judges can understand and experience in a 3-minute demo.

Challenge baseline from the brief: React + Mapbox GL JS + Highcharts/Chart.js on the frontend; Python (Flask/Django/FastAPI) + PostgreSQL/PostGIS + JWT on the backend; GitHub Actions, pre-commit quality checks, and public deployment are required. ■filecite■turn0file0■L25-L44■

1. PRODUCT STRATEGY

- **Core product:** a geospatial command center for managing projects, sites, and time-series environmental performance.
- **Demo promise:** “Create a project → draw a site → instantly understand its environmental performance.”
- **Primary judge journey:** Dashboard → Map → Draw Site → Site Intelligence → Time-series insight → project-level summary.
- **Scope rule:** implement the complete happy path and make secondary features visibly credible rather than building broad but shallow functionality.
- **Differentiation:** combine map-first interaction, polished analytics, environmental KPIs, and a believable production architecture in one coherent flow.

2. CORE FEATURES — MVP PRIORITY

Priority	Feature	What must work	Judge-visible wow
P0	Authentication	Login/register with JWT; seeded demo user	Fast, clean entry into the product.
P0	Project management	Create project; list projects; project status/KPIs	Shows real business workflow, not a static dashboard.
P0	Interactive geospatial map	Mapbox map with project/site markers and polygons	Immediate visual proof of geospatial capability.
P0	Draw site polygon	Draw/edit polygon; persist GeoJSON geometry	High-impact interaction; PostGIS can perform themselves.
P0	Site intelligence	Click site → KPI cards + metadata + chart	Transforms geometry into business insight.
P0	Time-series analytics	Carbon/biodiversity metrics over time with interactive chart	Makes the dashboard feel analytical rather than CRUD.
P1	Project summary	Aggregate area, carbon metric, biodiversity metrics, site count	Creates site-level narrative.
P1	Filters	Project/status/site filters affecting map and data	Signals scalable UX.
P1	Data quality state	Show loading, empty, error, and saved state	Production polish and reliability.
P2	Export/share	Optional screenshot/CSV export or shareable link	Only add if P0/P1 is stable.

3. TECHNICAL REQUIREMENTS

Frontend	React; responsive dashboard; componentized UI; Mapbox GL JS; Highcharts or Chart.js.
Backend	Python API using FastAPI/Flask/Django; REST endpoints for auth, projects, sites, and analytics.
Database	PostgreSQL + PostGIS; store project/site metadata and polygon geometry using a geospatial type.
Authentication	JWT-based authentication; protected API routes; demo account seeded for presentation.
Geospatial	Mapbox GL JS; drawing interaction; GeoJSON conversion; geometry persistence and retrieval.
Analytics	Chart.js or Highcharts; time-series carbon/biodiversity data; tooltips and responsive charts.
Quality	Automated lint/format/test checks; pre-commit hooks; clean error handling.
CI/CD	GitHub Actions pipeline; run checks on push/PR; deploy automatically to a public URL.

Documentation	README with architecture, schema, setup, CI/CD explanation, demo credentials/notes.
Deployment	Publicly accessible frontend/backend/database arrangement appropriate to the selected stack.

The brief explicitly requires clean architecture/schema, CI/CD, Mapbox and chart integration, and sensible technical trade-offs—not only visual output. [filecite turn0file0L59-L77](#)

4. RECOMMENDED SYSTEM ARCHITECTURE

- **Browser:** React SPA → API client → Mapbox/Chart.js rendering.
- **API:** Python service with JWT middleware → project/site/analytics services → PostgreSQL/PostGIS.
- **Data model:** User → Projects → Sites → Time-series metrics.
- **Geometry:** browser drawing → GeoJSON → API validation → PostGIS geometry → API response → Mapbox layer.
- **Analytics:** seeded metric records → aggregation endpoint → chart component.
- **Operational boundary:** keep environmental analytics deterministic for the hackathon; make the data model realistic enough that live/large datasets can be introduced later.

5. MINIMUM DATABASE SCHEMA

Table	Key fields	Purpose
users	id, email, password_hash, created_at	JWT identity and ownership.
projects	id, name, description, status, owner_id, created_at	Project master.
sites	id, project_id, name, area, geometry, status, created_at	Geospatial site master; geometry in PostGIS.
site_metrics	id, site_id, metric_date, carbon_value, biodiversity_value	Time-series analytics.

6. API CONTRACT — KEEP IT SMALL

POST	/auth/login	Return JWT
POST	/auth/register	Create user (optional if time constrained)
GET	/projects	List projects + summary KPIs
POST	/projects	Create project
GET	/projects/:id/sites	Return sites + GeoJSON
POST	/projects/:id/sites	Create site from polygon
GET	/sites/:id	Site metadata + KPI summary
GET	/sites/:id/metrics	Time-series analytics

7. 15-HOUR BUILD PLAN

0:00–0:45	Architecture + repo setup	Repo, env vars, folder structure, DB connection, basic CI config.
0:45–2:15	Backend foundation	FastAPI/Flask app, JWT, DB models, migrations/schema, health endpoint.
2:15–3:30	Project APIs	CRUD/list endpoints + seeded demo data + validation.
3:30–5:00	Map foundation	React shell, Mapbox setup, project/site layers, responsive map container.
5:00–6:30	Polygon drawing	Draw/edit polygon, calculate/display area, persist GeoJSON/PostGIS geometry.
6:30–8:00	Dashboard UX	Sidebar/top bar, project cards, KPI cards, map controls, filters.
8:00–9:30	Site intelligence	Site detail panel, metadata, KPIs, metrics API integration.
9:30–10:30	Charts	Time-series carbon/biodiversity chart, tooltips, trend labels, empty/loading states.
10:30–11:30	Polish pass	Micro-interactions, transitions, skeletons, toast feedback, responsive fixes.
11:30–12:15	Reliability	Error boundaries, API failure handling, demo seed/reset path, smoke tests.
12:15–13:00	CI/CD	GitHub Actions, lint/test/build, pre-commit, deploy and verify public URL.
13:00–14:00	Demo hardening	Run exact 3-minute script repeatedly; remove fragile interactions; seed final data.
14:00–15:00	README + final QA	Architecture/schema screenshots, setup, CI/CD, credentials, final browser/mobile check.

8. 3-MINUTE JUDGE DEMO SCRIPT

0:00–0:20 — Hook	Open directly on a populated dashboard. State the problem in one sentence and point to project-level environmental KPIs.
0:20–0:55 — Explore	Click a project. Show its sites on the Mapbox map. Use one filter to demonstrate the UI is data-driven.
0:55–1:35 — Build	Click “Add Site,” draw a polygon, finish it, and show area/site metadata. Save it.
1:35–2:20 — Understand	Open the new/site-selected intelligence panel. Show carbon + biodiversity KPI cards and the interactive time-series chart.
2:20–2:45 — Engineering	Briefly show the architecture: React → Python API → PostgreSQL/PostGIS, plus CI checks. Do not spend the demo clicking through code.
2:45–3:00 — Close	Return to dashboard and state the measurable product loop: project → site → geospatial data → analytics → decision support.

9. VISUAL / UX REQUIREMENTS

- **Map-first composition:** make the map the dominant visual surface, with analytics in a focused side panel or lower section.
- **Progressive disclosure:** show summary first; reveal detailed metrics only after site selection.
- **Animation budget:** 150–250 ms transitions for panels/cards; subtle map marker/polygon feedback; avoid animation that can delay interaction.
- **Visual hierarchy:** one primary CTA (“Add Site”), 3–4 top-level KPIs, one primary chart, clear status indicators.
- **Demo states:** pre-populate meaningful data so the first screen is never empty.
- **Resilience:** visible loading skeletons, API error toast, empty states, disabled buttons during save, and a confirmation after polygon creation.
- **Responsive:** desktop-first for judging, but ensure tablet/mobile does not collapse or overflow.

10. HACKATHON OPTIMIZATION — WHAT TO BUILD VS. SKIP

Build	End-to-end polygon creation and persistence; site intelligence; beautiful map; real API integration; seeded time-series; CI; public deployment; README.
Keep lightweight	Registration; project editing; filters; advanced permissions; exports.
Skip unless already working	Complex real-time collaboration, ML prediction, external data ingestion, elaborate admin roles, notifications, offline mode.

Never fake	Core map save flow, API calls, database persistence, authentication, and chart data integration. A small real system is stronger than a large simulated UI.
------------	---

11. PRODUCTION-READY PROTOTYPE CHECKLIST

- Authentication: JWT issued, protected routes verified, demo credentials documented.
- Project creation/listing works against the real database.
- Polygon can be drawn, edited, saved, retrieved, and rendered again after refresh.
- PostGIS geometry is stored with an explicit SRID and validated server-side.
- Site detail loads from API and chart data is API-backed.
- No console-breaking errors in the main demo journey.
- Loading/error/empty/success states exist for critical operations.
- Frontend build passes; backend tests/smoke tests pass.
- GitHub Actions runs automated quality checks.
- Pre-commit formatting/linting hooks are configured as required by the brief.
- Public deployment works from a clean browser session.
- README covers architecture, database schema, local setup, CI/CD, and required links/credentials. The submission brief explicitly calls for these items. ■filecite■turn0file0■L47-L58■
- Private repository access is granted to the specified hiring-team accounts if applicable. ■filecite■turn0file0■L88-L97■

12. FINAL DEMO HARDENING

- **Seed everything:** at least 2 projects, 4–6 sites, and 6–12 time-series points per site.
- **Make one path perfect:** the presenter should be able to execute the full journey without searching the UI.
- **Prepare a reset:** a single seed/reset mechanism should restore demo data if a live demo goes wrong.
- **Preflight:** open the deployed URL in an incognito window, log in, create a polygon, refresh, reopen it, and verify analytics.
- **Fallback:** if Mapbox token/API/network issues occur, have a seeded map state and clear recovery path; do not rely on a fragile last-minute external request.
- **README as evidence:** include one architecture diagram, schema diagram, API summary, setup steps, CI/CD explanation, and demo credentials.

13. THE “WOW” LAYER — ONLY AFTER CORE STABILITY

- **Live KPI transitions:** animate KPI values when changing project/site.
- **Map-to-chart linkage:** selecting a polygon updates the analytics panel instantly.
- **Environmental score:** derive a transparent composite indicator from the existing metrics; show the formula in a tooltip/info popover.
- **Trend narrative:** a short deterministic sentence such as “Carbon metric increased over the selected period” generated from the time-series data.
- **Mini-map legend:** visually encode site/project status without clutter.
- **Command-style shortcut:** one prominent “Add Site” action with keyboard accessibility if time permits.

14. SUCCESS CRITERIA

At the 3-minute mark, a reviewer should have seen: a polished React product, an interactive Mapbox map, real polygon drawing, persisted geospatial data, project/site management, interactive environmental analytics, a credible Python/PostGIS backend, and evidence of CI/CD/code-quality discipline.

The supplied brief states that evaluation is holistic across technical excellence and product mindedness, including code quality, scalable architecture/schema, CI/CD, correct use of Mapbox/charting, problem-solving, UI/UX, and trade-off reasoning. ■filecite■turn0file0■L59-L77■

Strategic principle: do not attempt to “bypass” competitors through gimmicks. In a 15-hour build, the practical advantage is a tighter scope, a more coherent user journey, stronger visual proof, and a genuinely working technical spine.